

Estrazione dei dati per fattore

Sul set di dati `ais` contenuto nel pacchetto **DAAG** – accertatevi di avere installato il pacchetto o in alternativa procedete come indicato in [1] – vogliamo eseguire una analisi statistica e grafica selezionando alcuni dati specifici: vediamo cosa contiene quindi lo impieghiamo in un esempio di **estrazione dei dati per fattore**.

Copiate e incollate nella Console di R questo script e premete ↵ Invio.

```
# ESTRAZIONE DEI DATI PER FATTORE
#
library(DAAG) # carica il pacchetto DAAG che include il set di dati ais
head(ais, 3) # mostra le prime tre righe del set di dati ais
tail(ais,3) # mostra le ultime tre righe del set di dati ais
str(ais) # mostra la struttura dei dati
#
```

Dopo avere caricato il pacchetto, con **head(ais, 3)** vediamo le prime tre righe di **ais**

```
> head(ais, 3) # mostra le prime tre righe del set di dati ais
  rcc wcc  hc  hg ferr  bmi  ssf pcBfat  lbm  ht  wt sex  sport
1 3.96 7.5 37.5 12.3  60 20.56 109.1  19.75 63.32 195.9 78.9  f B_Ball
2 4.41 8.3 38.2 12.7  68 20.67 102.8  21.30 58.55 189.7 74.4  f B_Ball
3 4.14 5.0 36.4 11.6  21 21.86 104.6  19.88 55.36 177.8 69.1  f B_Ball
```

con **tail(ais, 3)** vediamo le ultime tre righe

```
> tail(ais,3) # mostra le ultime tre righe del set di dati ais
  rcc wcc  hc  hg ferr  bmi  ssf pcBfat  lbm  ht  wt sex  sport
200 5.03 6.4 42.7 14.3  122 22.01 47.6   8.51  68 183.1 73.8  m Tennis
201 4.97 8.8 43.0 14.9  233 22.34 60.4  11.50  63 178.4 71.1  m Tennis
202 5.38 6.3 46.0 15.7   32 21.07 34.9   6.26  72 190.8 76.7  m Tennis
```

e con **str(ais)** vediamo la struttura dei dati contenuti in **ais**

```
> str(ais) # mostra la struttura dei dati
'data.frame': 202 obs. of 13 variables:
 $ rcc : num 3.96 4.41 4.14 4.11 4.45 4.1 4.31 4.42 4.3 4.51 ...
 $ wcc : num 7.5 8.3 5 5.3 6.8 4.4 5.3 5.7 8.9 4.4 ...
 $ hc : num 37.5 38.2 36.4 37.3 41.5 37.4 39.6 39.9 41.1 41.6 ...
 $ hg : num 12.3 12.7 11.6 12.6 14 12.5 12.8 13.2 13.5 12.7 ...
 $ ferr : num 60 68 21 69 29 42 73 44 41 44 ...
 $ bmi : num 20.6 20.7 21.9 21.9 19 ...
 $ ssf : num 109.1 102.8 104.6 126.4 80.3 ...
 $ pcBfat: num 19.8 21.3 19.9 23.7 17.6 ...
 $ lbm : num 63.3 58.5 55.4 57.2 53.2 ...
 $ ht : num 196 190 178 185 185 ...
 $ wt : num 78.9 74.4 69.1 74.9 64.6 63.7 75.2 62.3 66.5 62.9 ...
 $ sex : Factor w/ 2 levels "f","m": 1 1 1 1 1 1 1 1 1 1 ...
 $ sport : Factor w/ 10 levels "B_Ball","Field",...: 1 1 1 1 1 1 1 1 1 1 ...
```

Tutto molto chiaro e molto semplice, abbiamo 202 osservazioni (202 obs) di 13 variabili (13 variables) di cui:

→ 11 variabili numeriche (`num`), risultato di misure di dati ematologici e di dati biometrici effettuate in altrettanti atleti australiani [1];

→ due variabili "fattore" (`Factor`) cioè variabili qualitative che sono state impiegate per classificare gli atleti in base al sesso (`sex`) in due classi (2 levels) e in base allo sport praticato (`sport`) in dieci classi (10 levels).

Eccoci quindi al punto: prima di procedere con l'analisi statistica o grafica, vogliamo verificare in via preliminare se, nel caso della variabile emoglobina (`hg`), all'interno di ciascuno dei dieci sport praticati (`sport`) i dati sono distribuiti in modo normale (gaussiano), impiegando il *test di Shapiro-Wilk*.

Per farlo potremmo innanzitutto estrarre i valori di concentrazione dell'emoglobina (`ais$hg`) con la funzione `which()` per il primo sport (`ais$sport=='B_Ball'`) nell'elenco, impiegando questo codice

```
#  
BasketBall <- ais$hg[which(ais$sport=='B_Ball')]  
#
```

quindi dovremmo convertire i valori estratti in valori numerici

```
#  
BasketBall <- as.numeric(BasketBall)  
#
```

e infine potremmo applicare il test di Shapiro-Wilk

```
#  
shapiro.test(BasketBall)  
#
```

ottenendo questo risultato

```
Shapiro-Wilk normality test
```

```
data:  BasketBall  
W = 0.95839, p-value = 0.3833
```

E quanto sopra dovrebbe essere ripetuto per altre nove volte, cambiando manualmente il nome dello sport (e possibilmente per chiarezza anche quello della variabile nella quale i dati vengono estratti).

Fortunatamente esiste una strada molto più semplice: innanzitutto si preparano i dati impiegando la funzione `split()`, che consente di estrarre e separare automaticamente i valori della variabile `hg` della tabella `ais` (`ais$hg`) in base a tutti i valori assunti dal fattore sport (`ais$sport`), per farlo copiate e incollate nella Console di R questa riga di codice e premete ↵ Invio.

```
#  
# estrae e separa i valori di hg in base al fattore sport  
#  
hg_sport <- split(ais$hg, ais$sport)  
#
```

Per capire cosa è accaduto vediamo con la funzione `str()` la struttura dell'oggetto `hg_sport` così creato

```
#
# struttura dell'oggetto con i valori di hg separati in base al fattore sport
#
str(hg_sport)
#
```

che contiene ancora tutti i valori di concentrazione dell'emoglobina del set di dati **ais**, ma ora separati per ciascuno dei dieci sport praticati, con l'indicazione tra parentesi quadra del numero di dati per ciascuno di essi.

```
> str(hg_sport)
List of 10
 $ B_Ball : num [1:25] 12.3 12.7 11.6 12.6 14 12.5 12.8 13.2 13.5 12.7 ...
 $ Field  : num [1:19] 13.3 14.7 15.3 14.3 14.6 15 15.2 15.4 16.7 15.5 ...
 $ Gym    : num [1:4] 13.4 14 12.5 14.5
 $ Netball: num [1:23] 13.6 12.7 12.3 12.3 12.8 11.8 12.7 12.4 12.4 14.1 ...
 $ Row    : num [1:37] 13.9 14.7 13.3 12.9 14.7 14.3 14.5 13 12.5 14.5 ...
 $ Swim   : num [1:22] 13.8 13.3 12.9 12.7 14.4 14 13.9 14 13.1 15.9 ...
 $ T_400m : num [1:29] 15.9 12.4 14.7 13.9 13 13.8 13.2 12.6 13.5 13.7 ...
 $ T_Sprnt: num [1:15] 13.4 14.4 14.7 14.2 15.6 15.2 15.5 15.5 14.9 19.2 ...
 $ Tennis : num [1:11] 12 13.9 13.5 12.1 14.8 14.5 13.9 17.7 14.3 14.9 ...
 $ W_Polo : num [1:17] 14.4 15.2 15 16.2 15.6 16.2 14.8 15.8 14.4 15.8 ...
```

Il lavoro di preparazione dei dati è stato rapido, ma la cosa più interessante è che ora diventa rapido anche applicare il test di normalità a ciascuno dei dieci sport, è sufficiente infatti una sola riga di codice, copiatela e incollatela nella Console di R e premete ↵ Invio.

```
#
# test di normalità (gaussianità) di Shapiro-Wilk sui valori di hg per i dieci sport
#
sapply(hg_sport, shapiro.test)
#
```

Come si vede con la funzione **sapply()** il *test di Shapiro-Wilk* (**shapiro.test**) per la normalità (gaussianità) viene automaticamente applicato a ciascuno dei dieci sottoinsiemi ricavati dalla scomposizione dei valori di concentrazione dell'emoglobina in base al fattore sport e contenuti nell'oggetto **hg_sport**. Da notare per inciso che la distribuzione dei valori di emoglobina non si allontana mai in modo significativo da una distribuzione gaussiana (in nessun caso abbiamo $p < 0.05$)

```
> sapply(hg_sport, shapiro.test)
      B_Ball      Field
statistic 0.9583881 0.9740128
p.value   0.3833057 0.8528473
method    "Shapiro-Wilk normality test" "Shapiro-Wilk normality test"
data.name "X[[i]]"      "X[[i]]"
      Gym      Netball
statistic 0.9791217 0.95727
p.value   0.896848 0.4105184
method    "Shapiro-Wilk normality test" "Shapiro-Wilk normality test"
data.name "X[[i]]"      "X[[i]]"
      Row      Swim
statistic 0.9782721 0.9457531
p.value   0.6714365 0.2595768
method    "Shapiro-Wilk normality test" "Shapiro-Wilk normality test"
data.name "X[[i]]"      "X[[i]]"
      T_400m      T_Sprnt
```

```

statistic 0.9780738 0.8952224
p.value 0.7872988 0.08047871
method "Shapiro-Wilk normality test" "Shapiro-Wilk normality test"
data.name "X[[i]]" "X[[i]]"
Tennis W_Polo
statistic 0.9399817 0.9481069
p.value 0.520283 0.4272774
method "Shapiro-Wilk normality test" "Shapiro-Wilk normality test"
data.name "X[[i]]" "X[[i]]"

```

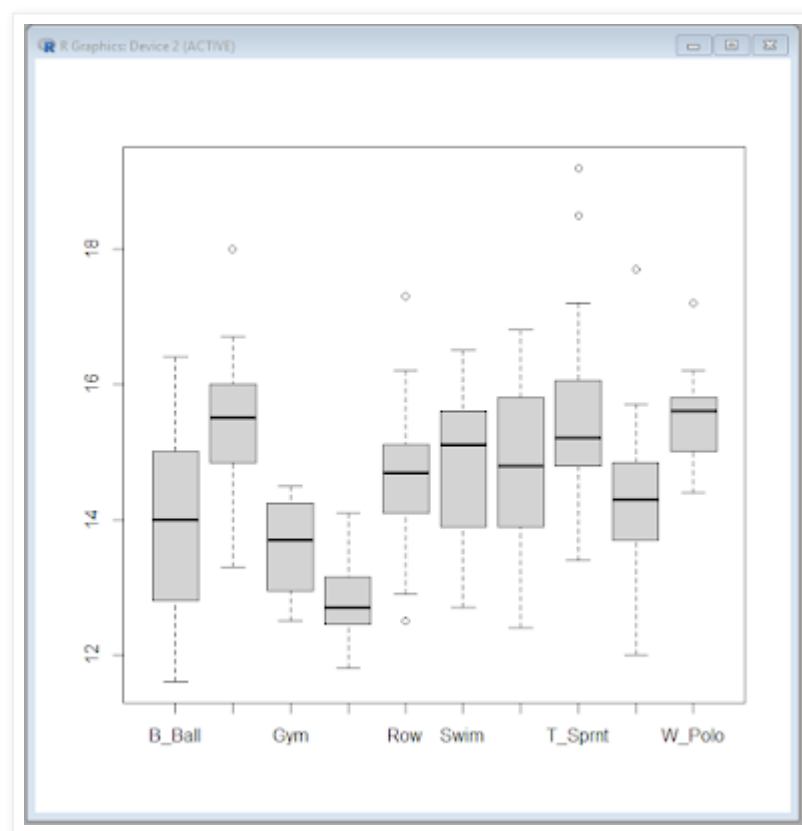
Gli stessi dati contenuti nell'oggetto **hg_sport** possono essere impiegati anche per generare (ad esempio) i grafici a scatola con i baffi delle dieci distribuzioni dei valori di concentrazione dell'emoglobina.

```

#
# boxplot dei valori di hg per i dieci sport
#
boxplot(hg_sport)
#

```

Ed ecco il grafico, in una versione base ma adeguata (per vedere i nomi di tutte le variabili, dopo avere generato il grafico potete "afferrare" con il mouse il lato sinistro della finestra e allargarla).



Ma non basta, perché al bisogno con la funzione **split()** è possibile separare i dati per più fattori: nel nostro caso possiamo separare i valori di concentrazione dell'emoglobina contemporaneamente per i due fattori sport e sesso.

```

#
# estrae e separa i valori di hg in base al fattore sport e al fattore sesso
#
hg_sport_sesso <- split(ais$hg, list(ais$sport, ais$sex), drop=TRUE)
#

```

Le cose da fare in questo caso sono:

→ impiegare la funzione **list()** per elencare i fattori in base ai quali i dati devono essere suddivisi, nel nostro caso sia lo sport praticato (**ais\$sport**) sia il sesso (**ais\$sex**);

→ aggiungere **drop=TRUE** per scartare i sottoinsiemi vuoti, nel nostro caso invece di 20 sottoinsiemi (10 sport x 2 sessi) avremo solamente 17 sottoinsiemi perché dei dieci sport le donne ne praticano 9 e gli uomini ne praticano 8.

Ora con la funzione **str()** si può avere la conferma che nell'oggetto **hg_sport_sesso** si trovano i 17 sottoinsiemi ricavati dalla scomposizione dei valori di concentrazione dell'emoglobina sia per sport sia per sesso:

```
> str(hg_sport_sesso)
```

List of 17

```
$ B_Ball.f : num [1:13] 12.3 12.7 11.6 12.6 14 12.5 12.8 13.2 13.5 12.7 ...
$ Field.f  : num [1:7] 13.3 14.7 15.3 14.3 14.6 15 15.2
$ Gym.f    : num [1:4] 13.4 14 12.5 14.5
$ Netball.f: num [1:23] 13.6 12.7 12.3 12.3 12.8 11.8 12.7 12.4 12.4 14.1 ...
$ Row.f    : num [1:22] 13.9 14.7 13.3 12.9 14.7 14.3 14.5 13 12.5 14.5 ...
$ Swim.f   : num [1:9] 13.8 13.3 12.9 12.7 14.4 14 13.9 14 13.1
$ T_400m.f : num [1:11] 15.9 12.4 14.7 13.9 13 13.8 13.2 12.6 13.5 13.7 ...
$ T_Sprnt.f: num [1:4] 13.4 14.4 14.7 14.2
$ Tennis.f : num [1:7] 12 13.9 13.5 12.1 14.8 14.5 13.9
$ B_Ball.m : num [1:12] 15.9 15.6 15.9 15.7 16.4 14.4 13.7 15.9 13.5 15 ...
$ Field.m  : num [1:12] 15.4 16.7 15.5 14.7 18 16.1 15.9 16.3 15.8 15.7 ...
$ Row.m    : num [1:15] 15 14.8 14.5 15.5 14.7 15.4 16.2 14.9 17.3 15.8 ...
$ Swim.m   : num [1:13] 15.9 15.2 15.9 15 15.6 15.2 16.3 15.2 16.5 15.4 ...
$ T_400m.m : num [1:18] 14.4 14 15.1 16.5 16.1 15 15.4 15.4 15.8 14.3 ...
$ T_Sprnt.m: num [1:11] 15.6 15.2 15.5 15.5 14.9 19.2 15.1 14.9 17.2 16.5 ...
$ Tennis.m : num [1:4] 17.7 14.3 14.9 15.7
$ W_Polo.m : num [1:17] 14.4 15.2 15 16.2 15.6 16.2 14.8 15.8 14.4 15.8 ...
```

ed è quindi possibile, automaticamente per tutti i sottoinsiemi, effettuare il test di normalità e rappresentare i boxplot con le funzioni di cui sopra, semplicemente sostituendo **hg_sport** con **hg_sport_sesso**.

La conclusione? Quando si inizia a lavorare con **R** molte volte ci si arena su problemi banali per i quali esistono soluzioni delle quali non si è a conoscenza data la vastità del linguaggio. Un esempio ne è l'estrazione dei dati per fattore da un set di dati mediante la funzione **split()**, che consente – indicando la variabile da estrarre e il fattore (o congiuntamente con **list()** i fattori) in base al quale separare i dati – di **preparare i dati** organizzandoli in un formato utile ai fini della loro successiva analisi statistica e della loro rappresentazione grafica.

[1] Vedere il post [Il set di dati ais](#). Nel post trovate anche come caricare i dati della tabella senza impiegare il pacchetto **DAAG**.